

PRACTICAL – 6

Rendering External JSON Data using Fetch API, Search and Filter

AIM:

To dynamically render event and student data on the StudentHub Events and Student Directory pages from external JSON files using the Fetch API, and to implement client-side search, category/department filtering, sorting and pagination using array methods, together with loading and error handling and a localStorage caching extension for offline-style fallback.

Introduction:

The StudentHub Events page, which previously displayed a fixed set of hard-coded event cards, was rebuilt to load its content from an external JSON file at runtime using the Fetch API. Three reusable JavaScript modules were added — `js/modules/api.js`, `js/modules/list-utils.js` and `js/modules/render.js` — along with two page-specific scripts, `js/events.js` and `js/students.js`, which power a new Student Directory page built from a second JSON dataset. Both pages share the same search, category filter, sort and pagination logic, and the same loading, error and offline-caching behaviour.

Tools and Technology:

VS Code, JavaScript (ES6 modules), the Fetch API, JSON, HTML5, CSS3, localStorage, browser developer tools (Console, Application/Storage), Git and GitHub, and a local development server (VS Code Live Server).

Datasets Used:

Dataset File	Records	Key Fields
data/events.json	16	id, title, category, date, location, description
data/students.json	16	id, name, department, semester, cgpa, email
data/faqs.json	16	id, category, question, answer (prepared as a third dataset)

Every dataset is loaded and cached through the same `fetchJson()` helper, and every list page reuses the same filter, sort, paginate and render functions, so the Events and Student Directory pages behave identically when loading, searching, sorting, paging through results, or recovering from a failed request.

Key Skills and Array Methods Used:

- `fetch()` and `async/await` to request the JSON files
- `try/catch` and `response.ok` to detect and handle network or server errors
- `Array.filter()` and `String.includes()` for the search box and category filter
- `Array.sort()` with `localeCompare()` and numeric comparison for the sort options
- `Array.slice()` to split the filtered, sorted list into pages
- `Array.map()` and template literals to render each record as an HTML card
- `Set` with `map()` to build a unique list of categories/departments for the dropdown
- `localStorage.setItem()/getItem()` to cache the last successful response

Implementation Overview:

1. Fetching and Rendering

A shared `fetchJson()` helper (`js/modules/api.js`) calls `fetch()` on the JSON URL, checks `response.ok`, parses the JSON, and returns the data. `js/events.js` and `js/students.js` call this helper and pass the result to `render.js`, which maps each record to an HTML card using the page's existing card styling.

2. Search and Category Filter

`filterItems()` (`js/modules/list-utils.js`) checks the search text against several fields (title, location, description for events; name, department, email for students) and checks the selected category/department, combining both conditions with `Array.filter()`.

3. Sorting

`sortItems()` reads the selected option (for example `date-asc`, `title-desc`, `cgpa-desc`) and sorts a copy of the filtered array using `Array.sort()`, switching between `localeCompare()` for text fields and numeric subtraction for number fields.

4. Pagination

`paginate()` slices the sorted array into pages of 6 records using `Array.slice()`. The page number is kept in a variable and re-applied whenever the search, filter or sort changes.

5. Loading and Error Handling

While the fetch is in progress, a "Loading..." message is shown. If the request fails (wrong URL, server down), a try/catch block catches the error; if a cached copy exists in `localStorage` it is used instead (with a visible notice), and if no cache exists a clear red error message is shown instead of a blank or broken page.

6. Modularity

The JavaScript is split into reusable modules: `api.js` (fetching and caching), `list-utils.js` (filter, sort, paginate — pure functions with no DOM code), and `render.js` (all DOM/HTML rendering). `events.js` and `students.js` are small page-specific files that import these modules and wire them to each page's controls.

7. Advanced Extension — Offline-style Caching

Every successful fetch is saved to `localStorage` under a page-specific key (`studenthub-events-cache`, `studenthub-students-cache`) along with a timestamp. If a later fetch fails, the cached copy is used automatically so the page still shows data instead of an empty screen.

Screenshots:

The JSON dataset, the search/filter/sort/pagination controls, the Student Directory page, the offline caching fallback, and the error state were all tested in the browser.

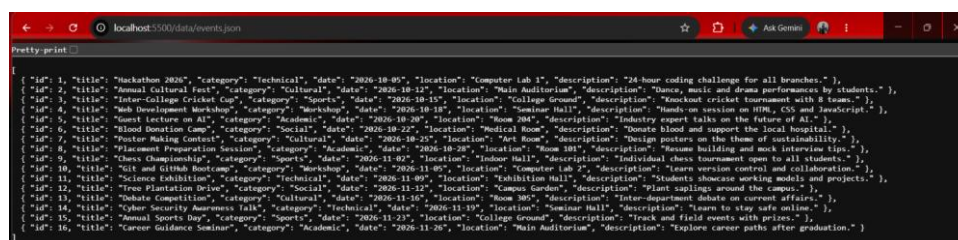


Figure 1: `data/events.json` opened directly in the browser, showing 16 valid JSON records.

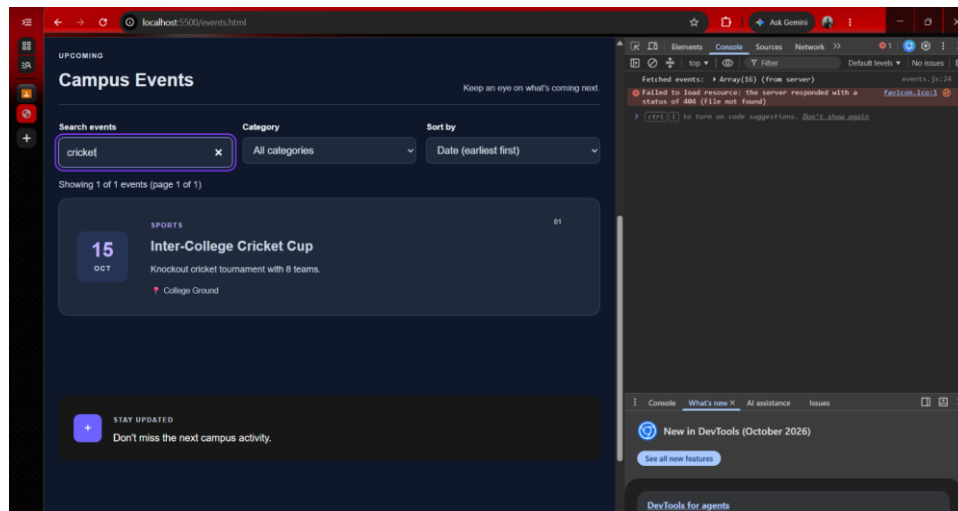


Figure 2: Search test — searching "cricket" correctly returns 1 of 16 events.

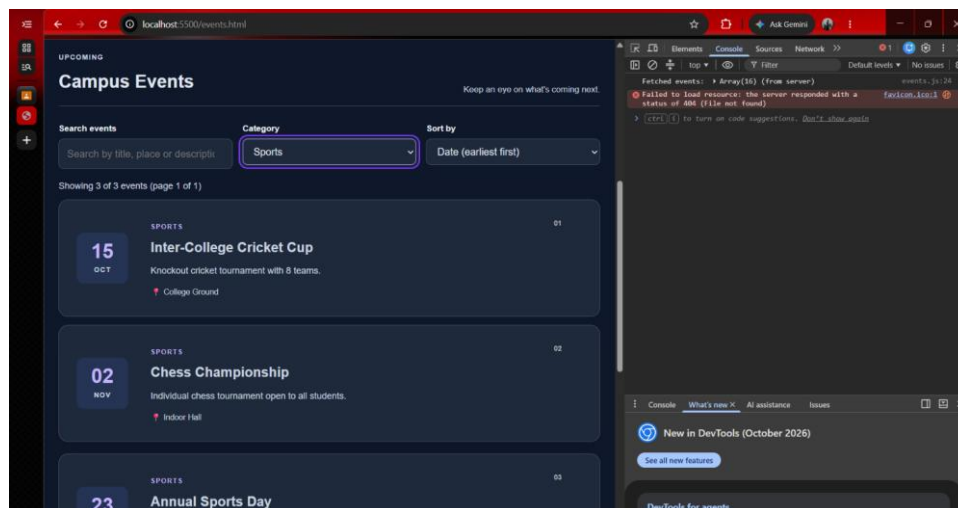


Figure 3: Category filter test — selecting "Sports" returns 3 of 16 events.

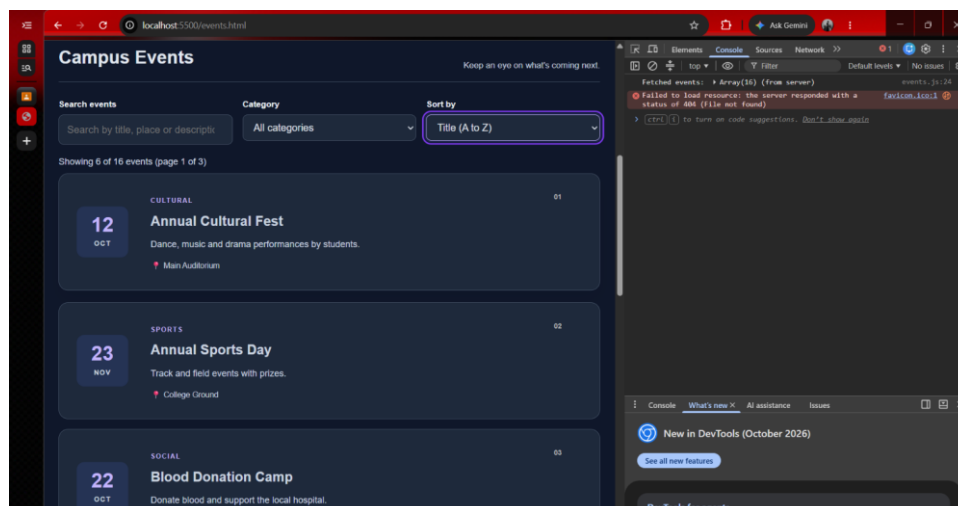


Figure 4: Sort test — "Title (A to Z)" places "Annual Cultural Fest" first.

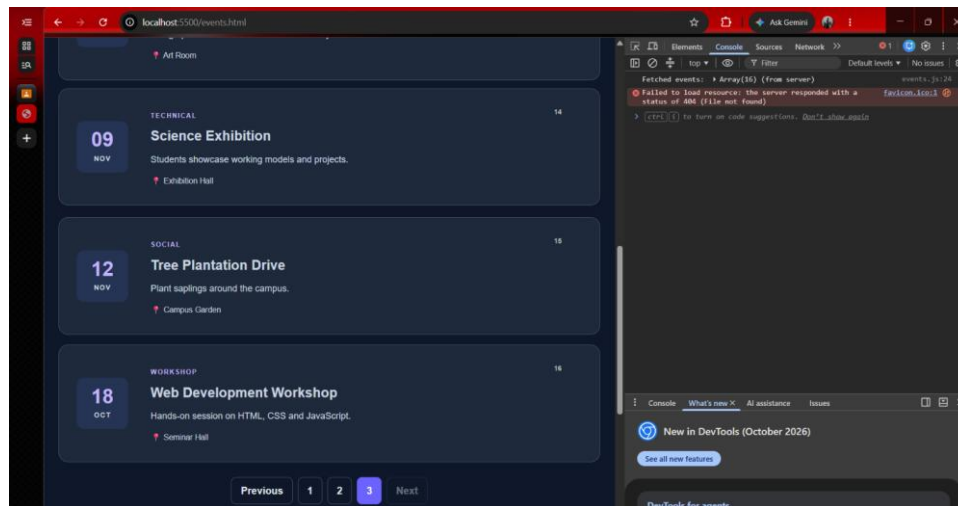


Figure 5: Pagination test — page 3 of 3 shows the last 4 events, with "Next" disabled.

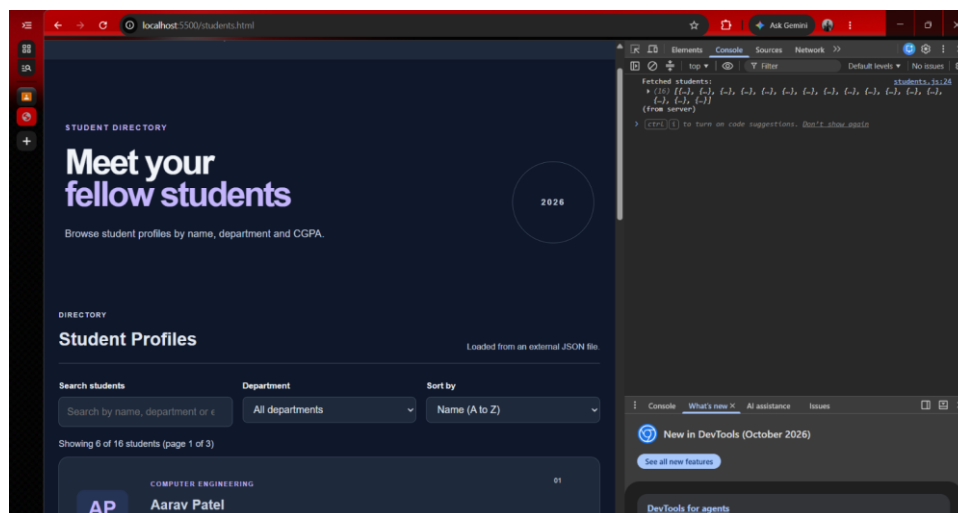


Figure 6: Student Directory page — students.html rendering all 16 records from data/students.json.

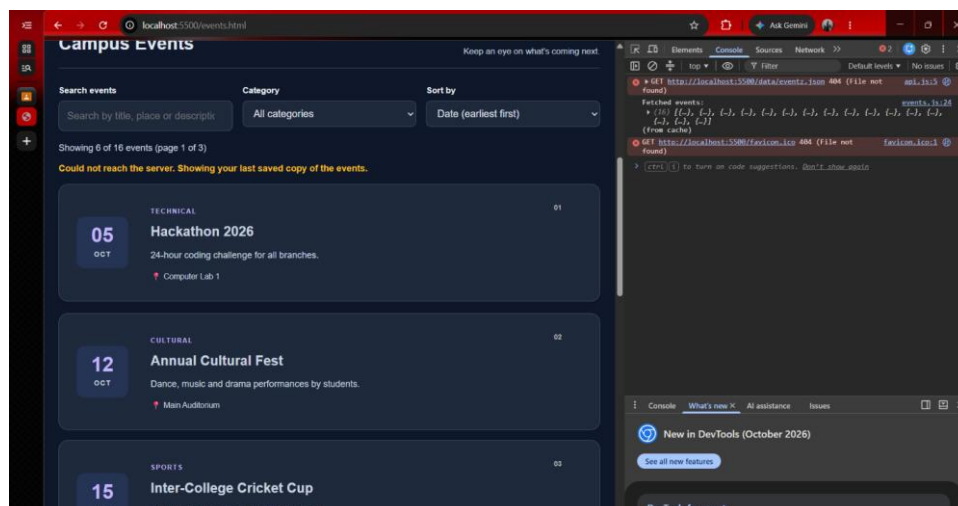


Figure 7: Cache fallback test — with the data URL intentionally broken, the page still shows all 16 events from the cached copy, with a visible notice and a 404 logged in the console.

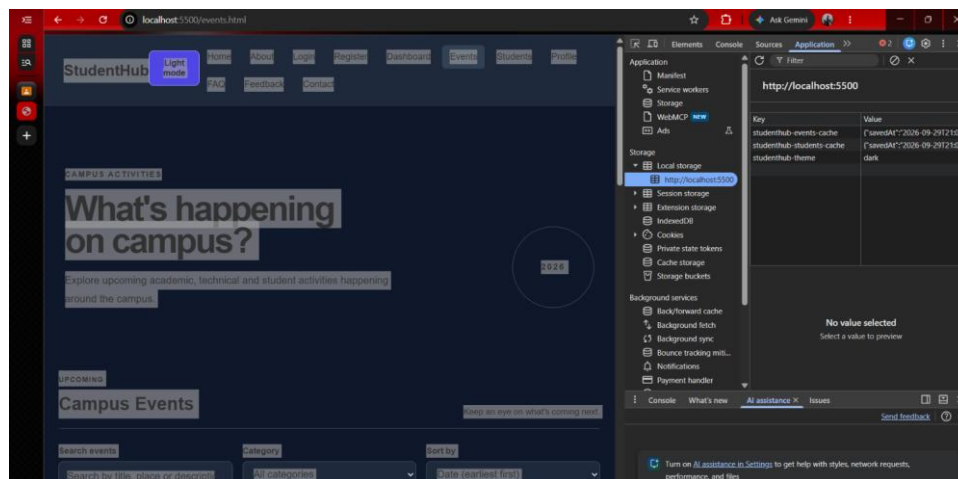


Figure 8: DevTools Application tab showing studenthub-events-cache and studenthub-students-cache saved in Local Storage.

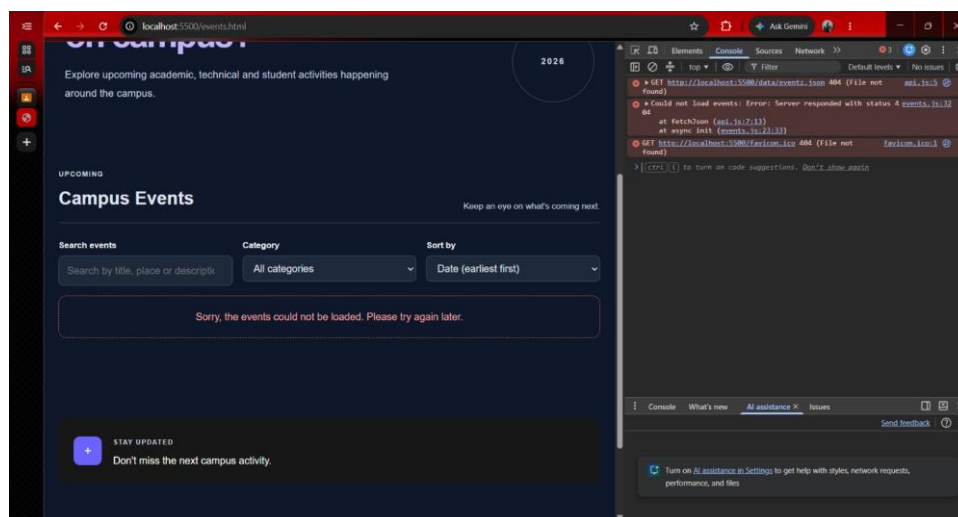


Figure 9: Error handling test — with the data URL broken and no cache available, a clear error message is shown instead of a broken page.

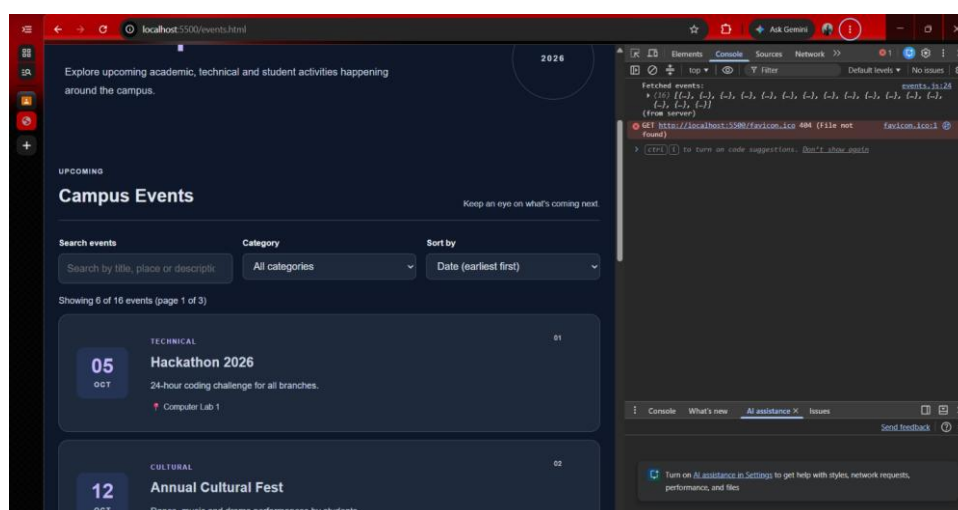


Figure 10: Final verification — after restoring the correct URL, the Events page loads normally again from the server.

Key Questions and Answers

Q1. How is JSON fetched, parsed and rendered?

`fetch()` requests the JSON file over HTTP. `response.json()` parses the response body into a JavaScript array. `Array.map()` then converts each record into an HTML string, which is inserted into the page with `innerHTML`.

Q2. Which array methods are used for search, filter, sort and pagination?

`filter()` and `includes()` for search and category matching, `sort()` with `localeCompare()`/numeric comparison for ordering, `slice()` to select one page of results, `map()` to render cards, and `Set` with `map()` to build the category/department dropdown.

Q3. How are loading and error states handled?

A "Loading..." message is shown while the fetch is in progress. A try/catch block, together with a check on `response.ok`, catches network and server errors. On failure the app first tries a cached copy from `localStorage`; if none exists, a clear error message is displayed instead of a blank or broken page.

Q4. How is modularity maintained in the JavaScript files?

The code is split by responsibility into ES modules: `api.js` handles fetching and caching, `list-utils.js` holds pure filter/sort/paginate functions, and `render.js` holds all DOM rendering. Each page (`events.js`, `students.js`) imports only what it needs from these shared modules.

Post Laboratory Work:

JSON files, rendered views, and test screenshots for search, filter, sort, pagination, loading, error handling and the caching extension were captured and are included above.

GITHUB REPOSITORY LINK:

<https://github.com/DarshilRupani/studenthub-portal>

(FOR SOURCE CODE)

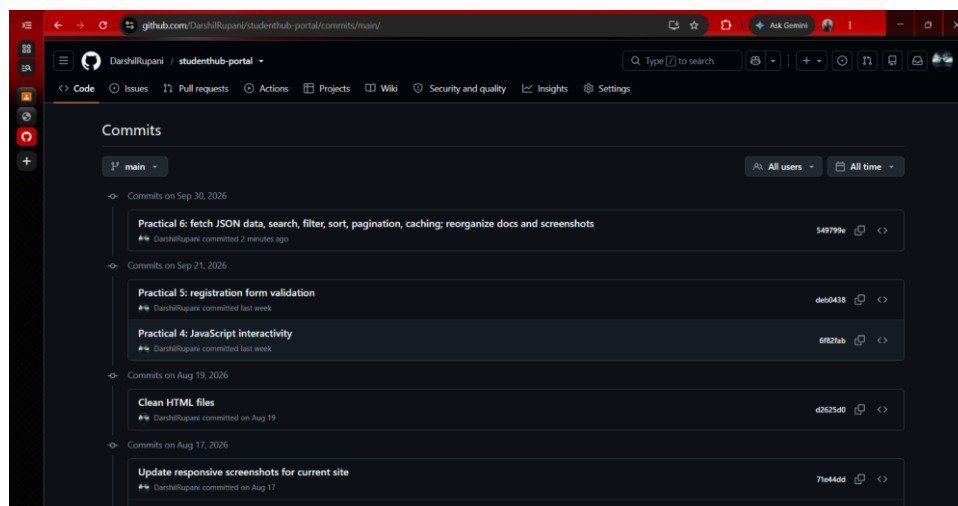


Figure 11: Practical 6 commit on GitHub.

Conclusion:

Practical 6 replaced hard-coded content on the Events page with data fetched from an external JSON file, and used the same pattern to build a new Student Directory page from a second JSON file. Search, category filtering, sorting and pagination were implemented using standard array methods, and the app was tested for both normal operation and failure conditions, including a working

localStorage caching extension that keeps the page usable when the network request fails. All required screenshots for search, filter, sort, pagination, error handling and caching are included above.